

AI 100 - Final Project Report

AI System Building with GenAI as a Cognitive Partner

Group Members	Nick Rishabh
LLM Used	Claude (claude-sonnet-4-6) by Anthropic
Project Task	3-Class Sentiment Classification using MLP + TF-IDF
Submission Date	May 1, 2026

Section I: What We Learned About GenAI

1.1 GenAI as a Socratic Partner, Not a Solution Machine

The most striking insight from this project was how differently Claude behaved when constrained to evaluate rather than solve. In normal use, asking Claude to fix a bug yields a corrected code block in seconds. In this project, the system prompt forced it into a Socratic role: evaluate the reflection depth first, label it Good or Bad, and only guide without solving when the reflection was weak. Every single one of our ten cases was labeled Bad, which meant we had to engage in genuine back-and-forth dialogue to deepen our own understanding.

This taught us that the helpfulness of a GenAI system is controllable through prompt design. A carefully constructed prompt can turn a powerful solution-generator into a tutor that enforces intellectual effort. The scaffolded prompt structure in the assignment was key: Claude consistently asked us to trace the precise failure point rather than accepting surface-level answers.

1.2 GenAI Surfaces the Why, Not Just the What

A concrete example: in Case 4, where we fitted TF-IDF on the test set, our initial reflection was that both sets still get vectorized so the model probably would not notice a difference. Claude's guidance asked us to think about what fit versus transform actually do, and what information leaks when vocabulary is derived from the test set. This redirected us from an output-level observation to a process-level understanding. That distinction is exactly what separates superficial debugging from principled machine learning practice.

Similarly in Case 8, where we shuffled texts and labels independently, our first reflection noted that the order still gets randomized, missing the point that independent shuffles produce different permutations. Claude's question about the probability that the i -th text still corresponds to the i -th label led us to articulate the combinatorial argument: with 600 items, the probability of the two shuffles aligning is astronomically small, guaranteeing mismatched pairs every time.

1.3 Limitations and Risks of Relying on GenAI

We also observed real limitations. In Case 6, where the activation function was changed to identity, our reflection was confidently wrong. Claude correctly labeled it Bad, but its Socratic question assumed we already understood the composition-of-linear-maps argument. Without prior exposure to linear algebra, the guidance would have been difficult to interpret. GenAI-as-tutor works best when the student has enough background to act on the hints it provides.

Another observation: Claude's guidance was occasionally too abstract. For Case 9, where `fl_score` was called with `average=None`, the hint about NumPy arrays not implementing the float formatting protocol was technically precise but required understanding Python's data model. This highlights that GenAI calibrates to an assumed background. Getting that calibration wrong can either under-help or overwhelm the student.

1.4 Prompt Engineering Is a Core Skill

We came to appreciate that composing the project's evaluation prompt was itself an act of engineering. The prompt had to be specific enough to prevent Claude from slipping into solution mode, yet open enough to allow nuanced guidance. Minor wording changes, such as replacing the word `explain` with `evaluate`, meaningfully shifted Claude's output style. This convinced us that understanding how to write effective prompts is as important as understanding how to write code when working in an AI-augmented environment.

Section II: What We Learned About Building AI Systems

2.1 Data Integrity Is the Foundation

Cases 2, 3, and 8 all demonstrated that bugs in the data pipeline are the most dangerous because they are silent. When we shuffled texts and labels independently in Case 8, the code ran without any error. Only the accuracy of roughly 27% revealed something was wrong. If we had expected low accuracy, we might have missed the bug entirely. Correct code syntax does not imply correct semantics, especially in data manipulation steps.

The lesson is to add explicit sanity checks after every data transformation step: verify that label distributions are preserved, that a sample text still matches its label, and that split sizes match expectations. Assertions and lightweight unit tests on data pipelines are not optional in production machine learning workflows.

2.2 The Train/Test Boundary Must Be Inviolable

Case 4 demonstrated one of the most common and most damaging mistakes in applied ML: data leakage through fitting a preprocessor on the test set. The code ran correctly and produced reasonable-looking accuracy numbers. Without an explicit audit of when `fit()` is called versus `transform()`, the leakage is invisible. This reinforced the principle that any statistic derived from training data, whether a vocabulary, a mean, a standard deviation, or PCA components, must be computed on training data only and then applied to dev and test sets without modification.

2.3 Hyperparameter Choices Have Principled Consequences

Cases 5, 6, and 7 showed that hyperparameter settings are not arbitrary knobs. Each has a mechanistic effect on what the model can represent and learn. Setting `max_features=1` in TF-IDF reduces the input to a single scalar, and no architecture can overcome that information bottleneck. Setting activation to identity collapses a three-layer MLP into a linear model regardless of depth. Setting `max_iter=1` freezes the model near random initialization. Understanding why these parameters matter, not just which values to use, is essential for effective debugging and model design.

2.4 Reproducibility Requires Explicit Seeding Throughout

Case 1, where the seed was changed from 42 to 0, produced high accuracy under both seeds, yet the results were not comparable across runs. In a research or production context, changing the seed without documenting it makes it impossible to determine whether a performance difference comes from a model improvement or a lucky data split. Best practice is to fix all seeds at every level of the stack, document them clearly, and run multiple seeds to estimate variance across different data partitions.

2.5 Error Messages Are Clues, Not Just Obstacles

Cases 9 and 10 produced immediate, explicit Python errors rather than silent misbehavior. At first these felt easier to diagnose than the silent bugs. However, understanding the error messages required knowing what type `f1_score` returns under different average settings in Case 9, and what sklearn's parameter validation rules are in Case 10. Reading error messages carefully and consulting the exact parameter contracts in documentation is a skill that accelerates debugging far more than trial-and-error guessing.

2.6 Evaluation Metrics Must Match the Task

Working through Cases 9 and 10 prompted us to think more carefully about evaluation design. Accuracy alone is insufficient for multi-class problems with potential class imbalance. The original code already reported Macro-F1, Micro-F1, precision, and recall. This project reinforced that choosing the right metric and computing it correctly is as important as choosing the right model. A bug in the evaluation code can make a bad model look good, or obscure genuine improvements entirely.